

FANTASYREALM ONLINE

Documentation technique

Architecture, environnement et choix technologiques

**Projet réalisé dans le cadre du Titre Studi
Développeur Web Gaming**

Rémi Michel

1. Présentation du projet

FantasyRealm Online est une application web permettant aux utilisateurs de créer, personnaliser, sauvegarder et partager des personnages issus d'un univers fantasy. Les personnages peuvent être soumis à validation avant leur publication dans un catalogue public. L'application propose également un système de commentaires, un espace employé destiné à la modération, la gestion des utilisateurs et du catalogue d'équipement ainsi qu'un espace administrateur permettant de gérer les paramètres applicatifs, les comptes employés et les journaux d'activité. Un formulaire de contact est également présent.

Le projet a été conçu comme une application web complète reposant sur une architecture Symfony, une base de données relationnelle MySQL et une base MongoDB. L'ensemble de l'environnement est conteneurisé avec Docker afin de faciliter l'installation locale et le déploiement.

2. Réflexion initiale et choix technologiques

Les choix technologiques ont été réalisés selon quatre objectifs : disposer d'une architecture maintenable, sécuriser les fonctionnalités sensibles, séparer les types de données selon leurs usages et garantir la reproductibilité de l'environnement de travail.

Technologie	Rôle	Justification
PHP / Symfony	Socle applicatif	Symfony fournit une architecture MVC structurée, l'injection de dépendances, le routing, les formulaires, la sécurité et de nombreux composants adaptés à une application métier.
Twig	Rendu des interfaces	Twig sépare la présentation de la logique applicative et facilite la création de vues réutilisables.
Doctrine ORM	Accès aux données MySQL	Doctrine permet de représenter le modèle relationnel sous forme d'entités PHP et de gérer les relations, migrations et requêtes.
MySQL	Données métier relationnelles	Les utilisateurs, personnages, équipements et commentaires possèdent des relations fortes et bénéficient d'un schéma structuré et de contraintes d'intégrité.
Doctrine MongoDB ODM	Accès aux documents MongoDB	L'ODM offre une intégration cohérente avec Symfony,
MongoDB	Journaux et paramètres	Les événements d'activité et paramètres applicatifs sont adaptés à un stockage documentaire, indépendant du modèle métier principal.
Bootstrap 5	Interface responsive	Bootstrap fournit une base responsive et accessible, complétée par une feuille de styles personnalisée.
Docker Compose	Environnement reproductible	Chaque service est isolé dans un conteneur, ce qui réduit les écarts de configuration entre les postes.

Mailpit	Tests des courriels	Les emails générés localement sont interceptés et consultables sans envoi réel, facilitant les phases de test.
Figma	Conception visuelle	Figma a été utilisé pour produire les wireframes et mockups desktop et mobile à l'aide de Figma Make.

3. Architecture générale

L'application est découpée en trois grosses briques : Symfony qui traite les requêtes HTTP et toute la logique métier grâce à son architecture MVC, MySQL pour les données métier sous format relationnel et MongoDB pour les données non relationnelles : logs d'activité et paramètres dynamiques.

Le code est organisé selon les conventions Symfony : contrôleurs, entités, documents MongoDB, formulaires, services, repositories, templates Twig et ressources publiques.

Des fixtures sont également utilisées afin de faciliter l'import de données initiales permettant de tester rapidement l'application.

4. Configuration de l'environnement de travail

Le développement a été réalisé sous Debian 13 avec Visual Studio. Le choix de l'OS et de l'IDE a été dictée selon mes habitudes de travail.

Le projet est exécuté dans Docker afin de garantir la reproductibilité de l'application peu importe l'environnement. Un test a été effectué sous Windows qui a été concluant.

Élément	Configuration	Utilisation
Système hôte	Debian 13	Poste principal de développement
Éditeur	Visual Studio	Édition PHP, Twig, YAML, CSS et Docker
Gestion de versions	Git / Github	Historique du code et partage du projet (utilisation de Git en ligne de commande et envoi sur compte Github personnel)
Conteneur PHP	PHP-FPM avec Composer	Exécution de Symfony et installation des dépendances
Serveur web	Nginx	Exposition de l'application en HTTP
Base relationnelle	MySQL 8.4	Stockage des données métier
Base documentaire	MongoDB	Journaux et configuration dynamique
Courriels locaux	Mailpit	Interception et consultation des emails
Conception	Figma Make	Wireframes et mockups

4.1 Services Docker

- **php** : Exécute PHP-FPM, Symfony, Composer et les commandes de console.
- **nginx** : Sert les fichiers publics et transmet les requêtes PHP au conteneur PHP.
- **db** : Héberge la base MySQL utilisée par Doctrine ORM.

- **mongodb** : Héberge la base documentaire utilisée par Doctrine MongoDB ODM.
- **mailpit** : Intercepte les messages envoyés par Symfony Mailer dans l'environnement local.
- **phpmyadmin** : Fournit une interface permettant rapidement de visualiser le contenu de la base MySQL.

4.2 Mise en route locale

Le démarrage standard s'effectue avec Docker Compose :

```
docker compose up -d --build
```

Les dépendances PHP, migrations et fixtures sont ensuite exécutées dans le conteneur PHP. Les variables d'environnement configurent notamment les connexions MySQL et MongoDB, le transport SMTP et le compte administrateur initial.

Tout est documenté dans le fichier README.md pour une mise en place locale rapide.

5. Organisation des données

5.1 Base relationnelle MySQL

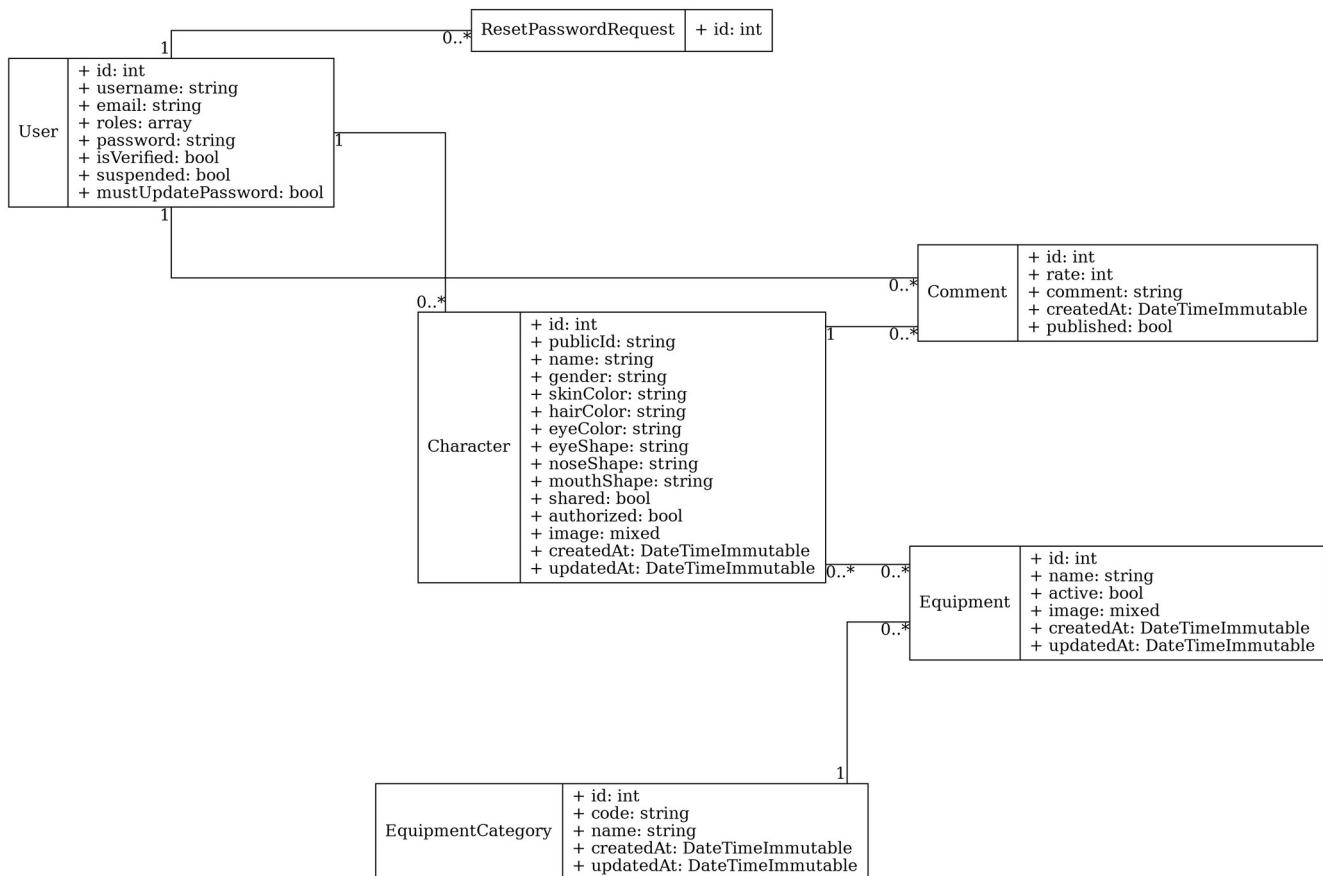
MySQL contient les données dont les relations et contraintes doivent être garanties : comptes utilisateurs, personnages, apparences, équipements, catégories et commentaires. Doctrine ORM assure le mapping entre les tables et les entités PHP.

5.2 Base documentaire MongoDB

MongoDB contient les journaux d'activité et les paramètres applicatifs. Etant donné qu'aucune relation n'est nécessaire pour ces éléments il s'agissait du meilleur choix pour répondre à la demande du cahier des charges d'utiliser deux types de base.

6. Modèle conceptuel / diagramme de classes

Le diagramme de classes présente les principales entités métier, leurs propriétés essentielles et leurs relations.



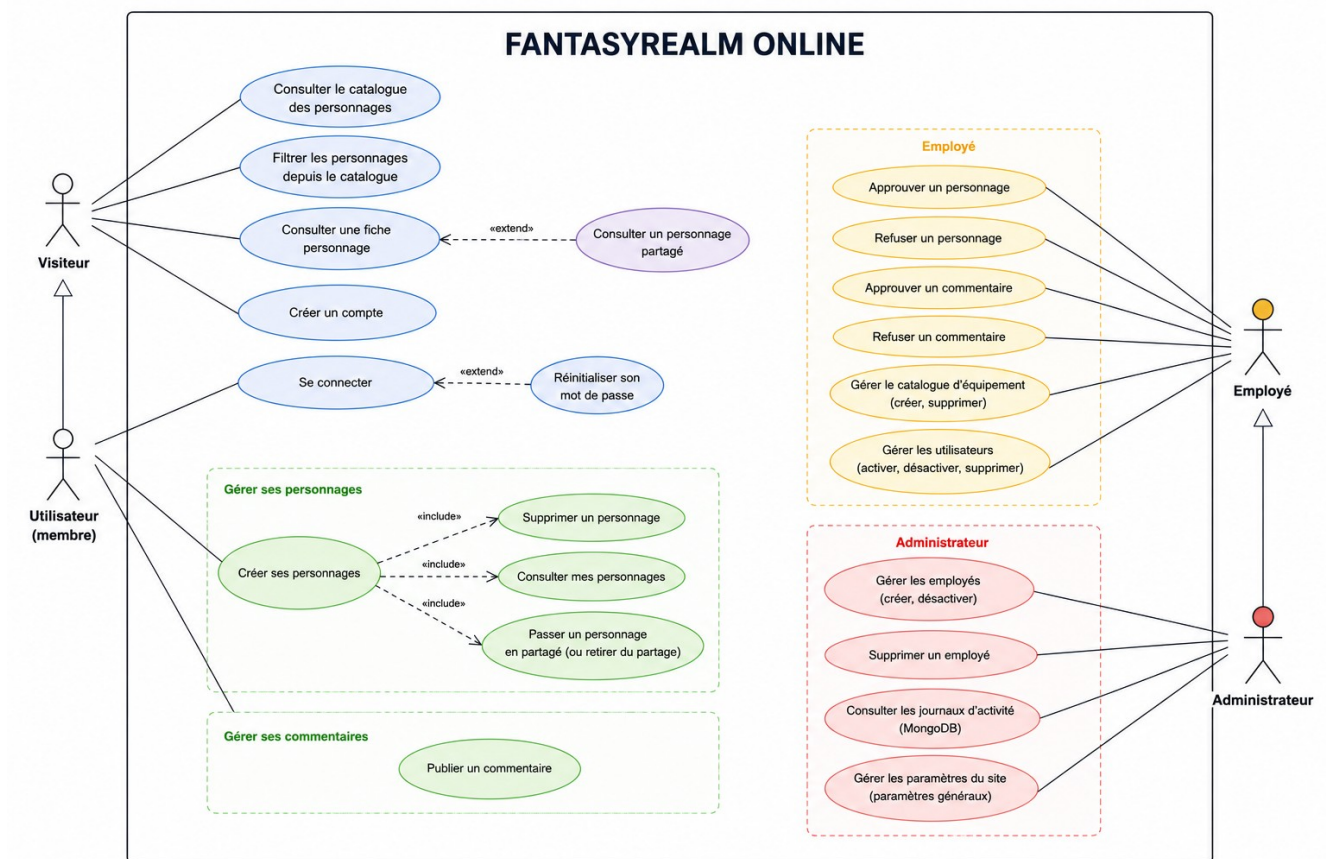
7. Principaux modules fonctionnels

- **Authentification et comptes** : Inscription, connexion, vérification, réinitialisation du mot de passe et gestion des rôles.
- **Créateur de personnage** : Saisie de l'identité, personnalisation visuelle, sélection des équipements et aperçu du résultat.
- **Catalogue public** : Consultation des personnages validés, filtres et accès aux fiches publiques contenant un dépôt d'avis utilisateurs.
- **Validation** : Traitement des personnages soumis et modération des commentaires par les employés ou administrateurs.
- **Administration** : Gestion du catalogue, des utilisateurs et pour les administrateurs uniquement des employés, des paramètres du site et des journaux d'activité.
- **Journalisation** : Création d'événements d'activité persistés dans MongoDB avec informations de contexte via un système de Factory.

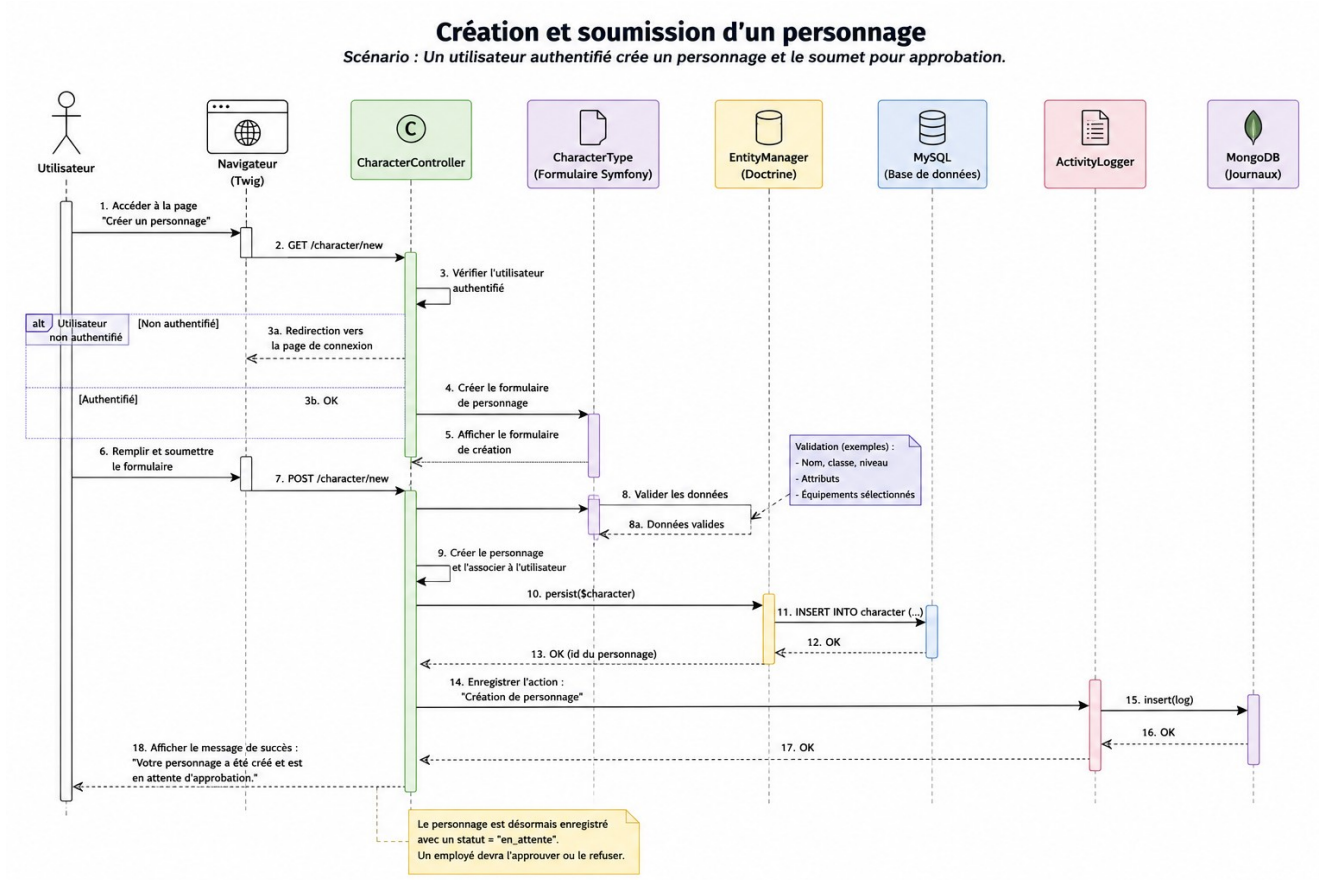
8. Sécurité et qualité

- Les mots de passe sont hachés avec le composant PasswordHasher de Symfony.
- Les accès sont contrôlés par rôles : ROLE_USER, ROLE_EMPLOYEE et ROLE_ADMIN.
- Les formulaires Symfony intègrent une protection CSRF.
- Les données saisies sont validées avant leur traitement.
- Les secrets et identifiants de production sont externalisés dans des variables d'environnement.
- Les bases MySQL et MongoDB communiquent avec l'application sur le réseau Docker interne.
- L'interface utilise une structure sémantique, des contrastes adaptés et une navigation responsive.

9. Diagramme d'utilisation



10. Diagramme de séquence



11. Déploiement de l'application

Le choix s'est porté sur un **VPS Debian** par OVH permettant d'héberger l'ensemble des services nécessaires au fonctionnement de l'application au sein de conteneurs Docker. L'application est désormais accessible via le domaine sécurisé :

<https://fantasyrealm-online.rmi-dev.tech>

Déploiement de l'application

La mise en production a été réalisée selon les étapes suivantes :

1. préparation du serveur Debian ;
2. installation de Docker et Docker Compose ;
3. récupération du projet depuis le dépôt GitHub ;

4. configuration des variables d'environnement de production ;
5. build démarrage des différents conteneurs Docker ;
6. exécution des migrations Doctrine afin de créer la structure de la base de données ;
7. chargement des données initiales à l'aide des fixtures ;
8. configuration du serveur Nginx ;
9. association du nom de domaine **fantasyrealm-online.rmi-dev.tech** au serveur ;
- 10.génération d'un certificat HTTPS Let's Encrypt ;
- 11.configuration du service SMTP Brevo pour l'envoi des courriels transactionnels ;
- 12.vérification du bon fonctionnement de l'application.

Grâce à Docker Compose, l'ensemble de ces opérations peut être reproduit facilement sur un autre serveur.

Configuration spécifique à la production

Quelques adaptations ont été nécessaires par rapport à l'environnement de développement :

- activation du mode **production** de Symfony ;
- utilisation d'un fichier de configuration spécifique (`.env.prod.local`) ;
- suppression de Mailpit au profit du service SMTP Brevo ;
- configuration d'un nom de domaine personnalisé ;
- mise en place d'un certificat HTTPS ;
- ouverture des ports HTTP (80) et HTTPS (443) sur le serveur ;
- authentification du domaine d'envoi auprès de Brevo afin de permettre l'envoi des courriels de confirmation et de réinitialisation de mot de passe.

Sécurité

Toutes les requêtes http sont automatiquement redirigées vers https depuis la configuration nginx.

L'envoi des emails est assuré par Brevo (renseigné en tant que MAILER_DSN côté Symfony).

Conclusion

Le recours à Docker Compose a considérablement simplifié le déploiement de l'application en garantissant une cohérence entre les environnements de développement et de production.

L'application est désormais entièrement opérationnelle dans un environnement de production sécurisé, accessible via un sous domaine personnalisé et utilisant des services réels pour la gestion des bases de données, des requêtes HTTPS et de l'envoi des emails.

12. Conclusion

L'architecture retenue permet de séparer clairement les responsabilités : Symfony orchestre la logique métier globale, MySQL garantit la cohérence des données métier, MongoDB stocke les documents techniques et Docker assure la portabilité de l'environnement. Cette organisation facilite la maintenance, les tests et le déploiement du site.